



UNIVERSITY REGISTRATION MANAGEMENT SYSTEM

Object Oriented Programming (Lab)



COURSE CODE: IT-106
SECTION: B

Submitted to: Mam Hijab Zainab

Submitted by: Laiba Saleem 25021556-096

Contents

INTRODUCTION	2
OBJECTIVES	2
PURPOSE OF THE PROJECT	2
SOFTWARE REQUIREMENT SPECIFICATION (SRS)	3
Functional Requirements	3
Non-Functional Requirements	3
SYSTEM ARCHITECTURE	3
1. Person Class	3
2. Student Class	4
3. Registration Class	4
4. University System Class	4
CLASS DIAGRAM	4
Use Case Diagram	5
MODULE DESCRIPTION	5
1. Add Student Module	5
2. Display Student Module	5
3. Search Student Module	5
4. Delete Student Module	6
5. Registration Module	6
ALGORITHMS	6
Add Student	6
Display Students	6
Search Student	6
Delete Student	7
Exit System	7
Code:	7
Output:	18

INTRODUCTION

The University Registration Management System is a console-based application developed in C++ using Object-Oriented Programming (OOP) concepts. The purpose of this project is to manage university student records in an efficient and organized way.

The system allows the user to add students, register courses, display records, search students, and delete records. This project demonstrates important OOP concepts including classes, objects, encapsulation, abstraction, inheritance, polymorphism, constructors, destructors, and dynamic memory allocation.

OBJECTIVES

- To develop a console-based management system using C++.
- To implement major Object-Oriented Programming concepts.
- To manage university student records efficiently.
- To improve programming and problem-solving skills.
- To understand inheritance and polymorphism practically.

PURPOSE OF THE PROJECT

The main purpose of this project is to automate the manual process of university registration management. The system helps in storing and managing student records quickly and efficiently. It reduces paperwork and improves data handling.

SOFTWARE REQUIREMENT SPECIFICATION (SRS)

Functional Requirements

The system should be able to:

1. Add Student Record
2. Display Student Records
3. Search Student by ID
4. Delete Student Record
5. Register Course information
6. Display Fee Information
7. Exit Program Safely

Non-Functional Requirements

- User-friendly console interface
- Fast execution
- Efficient memory management
- Reusable code structure
- Simple and understandable system

SYSTEM ARCHITECTURE

Classes Used

1. Person Class

Stores general person information.

2. Student Class

Derived from Person class and stores student details.

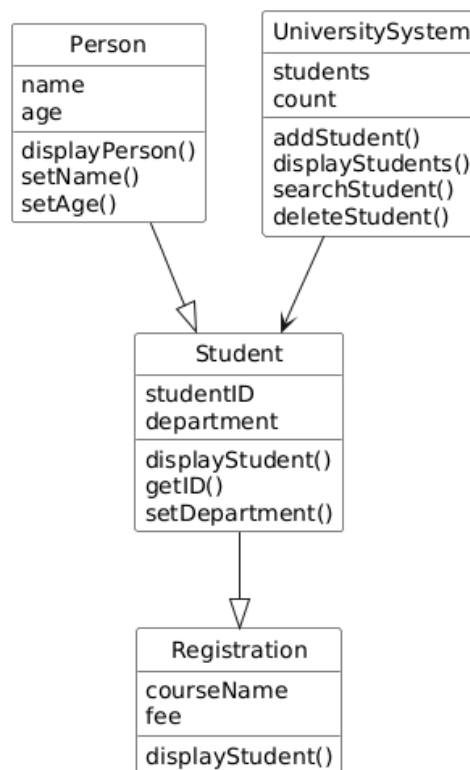
3. Registration Class

Stores course registration details.

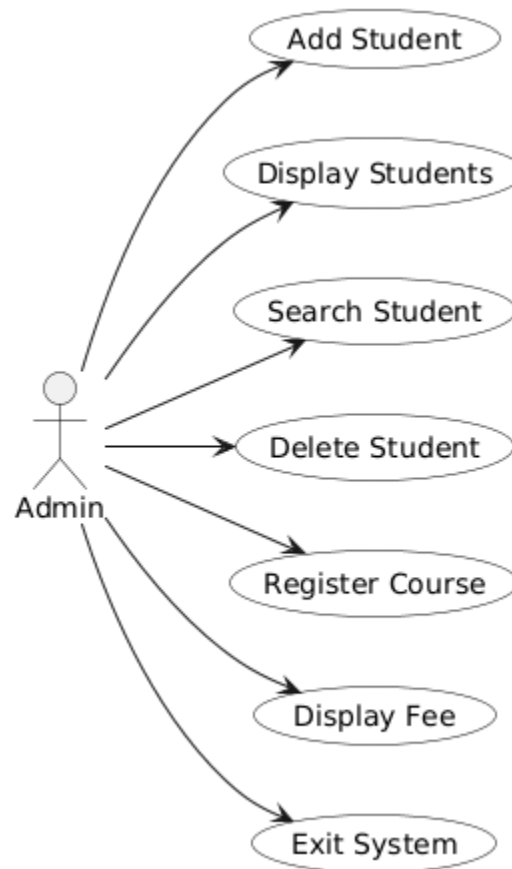
4. University System Class

Controls the complete system.

CLASS DIAGRAM



Use Case Diagram



MODULE DESCRIPTION

1. Add Student Module

This module is used to add new student records into the system.

2. Display Student Module

This module displays all available student records.

3. Search Student Module

This module searches a student record using student ID.

4. Delete Student Module

This module deletes a student record from the system.

5. Registration Module

This module stores course registration details and fee information.

ALGORITHMS

Add Student

- Start Program
- Select “Add Student” option
- Enter student details
- Create student object dynamically
- Store object in array
- Display success message
- End

Display Students

- Select “Display Students” option
- Check if records exist
- If records exist, display all student information
- Otherwise display “No Records Found”
- End

Search Student

- Select “Search Student” option
- Enter student ID
- Compare entered ID with stored records
- If record found, display student information

- Otherwise display “Student Not Found”
- End

Delete Student

- Select “Delete Student” option
- Enter student ID
- Search student record
- Delete object using delete operator
- Shift remaining records
- Display success message
- End

Exit System

- Select “Exit” option
- Release allocated memory
- Close the program
- End

Code:

```
#include<iostream>

#include<string>

using namespace std;

class Person
{
protected:
    string name;

    int age;
```



```
public:
    Person()
    {
        name = "";
        age = 0;
    }
    Person(string n, int a)
    {
        name = n;
        age = a;
    }
    virtual void displayPerson()
    {
        cout << "\nName: " << name;
        cout << "\nAge: " << age;
    }
    virtual ~Person() {}
};

class Student : public Person
{
protected:
    int studentID;
    string department;
```

public:

```
Student() : Person()
```

```
{
```

```
    studentID = 0;
```

```
    department = "";
```

```
}
```

```
Student(string n, int a, int id, string dept)
```

```
    : Person(n, a)
```

```
{
```

```
    studentID = id;
```

```
    department = dept;
```

```
}
```

```
int getID()
```

```
{
```

```
    return studentID;
```

```
}
```

```
virtual void displayStudent()
```

```
{
```

```
    displayPerson();
```

```
    cout << "\nStudent ID: " << studentID;
```

```
    cout << "\nDepartment: " << department;
```

```
}
```

```
virtual ~Student() {}  
};  
class Registration : public Student  
{  
private:  
    string courseName;  
    float fee;  
public:  
    Registration(string n, int a, int id,  
        string dept, string course, float f)  
        : Student(n, a, id, dept)  
    {  
        courseName = course;  
        fee = f;  
    }  
  
    void displayStudent()  
    {  
        cout << "\n\nSTUDENT RECORD";  
        cout << "\nName : " << name;  
        cout << "\nAge : " << age;  
        cout << "\nStudent ID : " << studentID;  
        cout << "\nDepartment : " << department;
```

```
        cout << "\nCourse : " << courseName;

        cout << "\nFee : " << fee << endl;

    }

};

class UniversitySystem
{
private:
    Student* students[100];

    int count;

public:
    UniversitySystem()
    {
        count = 0;

        for(int i = 0; i < 100; i++)
        {
            students[i] = NULL;
        }
    }

    void addStudent()
    {
        if(count >= 100)
        {
            cout << "\nLimit reached!\n";
```

```
        return;
    }

    string name, dept, course;
    int age, id;
    float fee;
    cin.ignore();
    cout << "\nEnter Name: ";
    getline(cin, name);
    cout << "Enter Age: ";
    cin >> age;
    cout << "Enter ID: ";
    cin >> id;
    cin.ignore();
    cout << "Enter Department: ";
    getline(cin, dept);
    cout << "Enter Course: ";
    getline(cin, course);
    cout << "Enter Fee: ";
    cin >> fee;

    students[count] = new Registration(name, age, id, dept, course, fee);
    count++;
    cout << "\nStudent Added Successfully!\n";
}
```

```
void displayStudents()
{
    if(count == 0)
    {
        cout << "\nNo Records Found!\n";
        return;
    }
    for(int i = 0; i < count; i++)
    {
        if(students[i] != NULL)
        {
            students[i]->displayStudent();
        }
    }
}

void searchStudent()
{
    int id;
    bool found = false;
    cout << "\nEnter Student ID: ";
    cin >> id;
    for(int i = 0; i < count; i++)
    {
```

```

        if(students[i] != NULL && students[i]->getID() == id)
        {
            students[i]->displayStudent();

            found = true;

            break;
        }
    }

    if(!found)
    {
        cout << "\nStudent Not Found!\n";
    }
}

void deleteStudent()
{
    int id;

    bool found = false;

    cout << "\nEnter Student ID: ";

    cin >> id;

    for(int i = 0; i < count; i++)
    {
        if(students[i] != NULL && students[i]->getID() == id)
        {

```

```
        delete students[i];
        for(int j = i; j < count - 1; j++)
        {
            students[j] = students[j + 1];
        }

        count--;
        students[count] = NULL;
        found = true;
        cout << "\nRecord Deleted Successfully!\n";

        break;
    }
}
if(!found)
{
    cout << "\nStudent Not Found!\n";
}
}
~UniversitySystem()
{
    for(int i = 0; i < count; i++)
    {
```



```
        delete students[i];
        students[i] = NULL;
    }
}
};

int main()
{
    UniversitySystem system;
    int choice;
    do
    {
        cout << "\n\nUNIVERSITY REGISTRATION SYSTEM";
        cout << "\n1. Add Student";
        cout << "\n2. Display Students";
        cout << "\n3. Search Student";
        cout << "\n4. Delete Student";
        cout << "\n5. Exit";
        cout << "\nEnter Choice: ";
        cin >> choice;
        switch(choice)
        {
            case 1:
                system.addStudent();
```

```
        break;
    case 2:
        system.displayStudents();
        break;
    case 3:
        system.searchStudent();
        break;
    case 4:
        system.deleteStudent();
        break;
    case 5:
        cout << "\nProgram Ended Successfully!\n";
        break;
    default:
        cout << "\nInvalid Choice!\n";
    }
} while(choice != 5);
return 0;
}
```

Output:

```
UNIVERSITY REGISTRATION SYSTEM
1. Add Student
2. Display Students
3. Search Student
4. Delete Student
5. Exit
```

```
Enter Choice: 1
```

```
Enter Name: Aisha
Enter Age: 19
Enter ID: 029
Enter Department: IT
Enter Course: OOP
Enter Fee: 85000
```

```
Student Added Successfully!
```

```
Enter Choice: 2
```

```
STUDENT RECORD
Name      : Aisha
Age       : 19
Student ID : 29
Department : IT
Course    : OOP
Fee       : 85000
```

```
UNIVERSITY REGISTRATION SYSTEM
1. Add Student
2. Display Students
3. Search Student
4. Delete Student
5. Exit
```

```
Enter Choice: 3

Enter Student ID: 029

STUDENT RECORD
Name      : Aisha
Age       : 19
Student ID : 29
Department : IT
Course    : OOP
Fee       : 85000

UNIVERSITY REGISTRATION SYSTEM
1. Add Student
2. Display Students
3. Search Student
4. Delete Student
5. Exit
```

```
Enter Choice: 4

Enter Student ID: 29

Record Deleted Successfully!

UNIVERSITY REGISTRATION SYSTEM
1. Add Student
2. Display Students
3. Search Student
4. Delete Student
5. Exit

Enter Choice: 5

Program Ended Successfully!

-----
Process exited after 104.9 seconds with return value 0
Press any key to continue . . .
```

THANK YOU
